

ClubHouse

Multi-College Student Collaboration Platform

FULL TECHNICAL DOCUMENTATION — Every Detail

Item	Detail
Workspace	D:\OpencodeProjs\Clubhouse\Web-Mobile-Fit
Date	20 September 2026
Stack	React 19 + Vite 7 + Express 5 + PostgreSQL + Drizzle + Clerk + Tailwind v4 + pnpm
Packages	3 artifacts (api-server, clubhouse, mockup-sandbox) + 5 libs + scripts
API contract	OpenAPI 3.1 — 47 paths / 64 operations
Database	17 tables (PostgreSQL + Drizzle ORM)
Auth	Clerk (frontend @clerk/react 6, backend @clerk/express 2)
Source of truth	replit.md + lib/db schema + lib/api-spec/openapi.yaml

Generated by automated static analysis of the repository (source files, package manifests, OpenAPI spec, DB schema, routes, pages). No runtime probing. Covers architecture, frontend, backend, database, API, auth, storage, config, operations, user flows, roles, design system, decisions, gotchas and roadmap.

Admin bootstrap: harshavardhankalvir2808@gmail.com | Brand primary: indigo 245 80% 58% | API port 8080, Web dev port 21903

Table of Contents

- 1. Executive Summary & Product Vision
- 2. Monorepo & Workspace Architecture
- 3. Technology Stack (Complete)
- 4. Frontend — clubhouse (React + Vite)
- 5. Backend — api-server (Express 5)
- 6. Database Schema — 17 Tables
- 7. API Contract — OpenAPI + Codegen
- 8. Authentication & Authorization (Clerk)
- 9. Object Storage & File Uploads
- 10. Styling, Design System & UI Kit
- 11. Configuration, Env Vars, Ports & Commands
- 12. Build, Typecheck, Deploy (Replit)
- 13. User Flows & Product Features
- 14. Roles & Permissions Matrix
- 15. Architecture Decisions & Gotchas

- 16. Repository Health & Git History
- 17. Roadmap & Improvement Recommendations
- Appendix A — Directory Tree & Cheat Sheet

1. Executive Summary & Product Vision

ClubHouse is a multi-college student collaboration platform. Students register, complete a 2-step onboarding profile (name, age, course, semester, college, pronouns, bio, portfolio, socials), join their college club, browse/discover projects, apply or get invited, collaborate via project-scoped real-time chat (polling) and meeting calendars, attend hackathons / workshops / seminars, and are moderated by college moderators and platform admins. Admins approve college registrations and moderator applications and view platform stats.

Contract-first engineering is the core philosophy: **lib/api-spec/openapi.yaml** is the single source of truth for the HTTP contract. Orval generates typed React-Query hooks (**lib/api-client-react**) and Zod validators (**lib/api-zod**). The frontend contains **zero hand-written fetch calls**. Similarly **lib/db/src/schema** is the single source of truth for persistence. Clerk is the sole identity provider — no custom OTP/password logic.

The repo is a pnpm workspace (Node 24, TS 5.9, React 19.1.0 pinned for Expo compat, Tailwind v4). It ships two deployables: **@workspace/api-server** (Express 5, port 8080, mounted at /api) and **@workspace/clubhouse** (Vite SPA, dev port 21903, static dist/public), plus an isolated **mockup-sandbox** for UI experiments.

2. Monorepo & Workspace Architecture

Root files: package.json (private workspace, enforces pnpm via preinstall, scripts build/typecheck), pnpm-workspace.yaml (packages: artifacts/*, lib/*, lib/integrations/*, scripts; catalog pinning; minimumReleaseAge 1440 supply-chain buffer; onlyBuiltDependencies; esbuild/lightningcss/rollup platform stripping), tsconfig.base.json (es2022/esnext, bundler resolution, strict, workspace conditions), tsconfig.json (project references to lib/db, lib/api-client-react, lib/api-zod).

Package	Type / Entry	Key notes
artifacts/api-server @workspace/api-server	Deployable API src/index.ts → app.ts	Express 5, esbuild ESM bundle dist/index.mjs, pino logging, Clerk middleware
artifacts/clubhouse @workspace/clubhouse	Deployable SPA src/main.tsx → App.tsx	React 19, Vite 7, wouter, TanStack Query, shadcn, Tailwind v4
artifacts/mockup-sandbox @workspace/mockup-sandbox 2.0.0	Isolated UI sandbox	Same Radix/Tailwind stack, chokidar+fast-glob, preview plugin
lib/db @workspace/db	Library src/index.ts + /schema	Drizzle + pg Pool, drizzle-kit push, exports . and ./schema
lib/api-spec @workspace/api-spec	Spec only openapi.yaml 1963 lines	Orval codegen source, v0.1.0, servers [/api]
lib/api-client-react @workspace/api-client-react	Library src/index.ts	Generated React-Query hooks, customFetch mutator, mode split
lib/api-zod @workspace/api-zod	Library src/index.ts	Generated Zod schemas, coercion + dates/bigint
lib/object-storage-web @workspace/object-storage-web	Library ObjectUploader.tsx	Uppy aws-s3/core/dashboard/react v5, peer react>=18
scripts @workspace/scripts	Tooling src/hello.ts	tsx runner, demo hello script

Dependency flow: clubhouse → api-client-react + object-storage-web; api-server → db + api-zod; api-spec → (codegen) → api-client-react + api-zod. TypeScript project references enforce build order. Root build = typecheck libs then pnpm -r build.

3. Technology Stack (Complete)

Layer	Choice / Version	Details
Language / runtime	TypeScript ~5.9.3, Node.js 24	strict, bundler resolution, isolatedModules, composite libs
Package manager	pnpm workspaces + catalog	autoInstallPeers false, minimumReleaseAge 1440, catalog pins react 19.1.0, vite 7.3.2, zod 3.25.76, drizzle 0.45.2
Frontend	React 19.1 + React-DOM 19.1, Vite 7.3	wouter 3.3.5 routing, TanStack Query 5.90, react-hook-form + resolvers
UI kit	shadcn/new-york + Radix + Tailwind v4.1	60+ ui/* components, next-themes, lucide-react, framer-motion, cmdk, recharts, sonner, vaul
Backend	Express 5.2.1	cookie-parser, cors, pino + pino-http, http-proxy-middleware
Database	PostgreSQL 16 + Drizzle ORM 0.45 + pg 8.22	drizzle-zod 0.8.3, drizzle-kit 0.31 push
Auth	Clerk (@clerk/react 6.11, @clerk/themes, @clerk/express 2.1, @clerk/shared)	ClerkProvider + clerkMiddleware, proxy /api/ __clerk
Validation	Zod v3/v4	Generated Zod via Orval, drizzle-zod for inserts
API codegen	Orval 8.18	react-query split client + zod client, baseUrl /api
Build	Vite build (web), esbuild 0.27.3 ESM (api)	esbuild-plugin-pino, source-maps, externals for native SDKs
Storage	Google Cloud Storage presigned PUT	@google-cloud/storage, Uppy uploader
Styling	Tailwind v4 (@tailwindcss/vite, typography, tw-animate)	HSL tokens, dark variant, Inter/JetBrains Mono fonts

4. Frontend — clubhouse (React + Vite)

Entry: src/main.tsx (createRoot) → App.tsx. Providers: WouterRouter (base=BASE_URL), QueryClientProvider (retry 1, staleTime 30s), ThemeProvider (class, default light), TooltipProvider, Toaster, ClerkProvider (publishableKeyFromHost + proxyUrl + signIn/Up URLs + routerPush/Replace that strip base + clerkAppearance). No hand fetch — all data via Orval hooks (useListClubs, useGetCollege, useJoinCollege, useSendMessage, etc.) + customFetch.

Routing (wouter)

Route	Component (size)	Purpose
/	Landing (7.5k)	Public marketing hero
/sign-in/*	SignInPage (inline Clerk SignIn)	routing=path, fallback /dashboard
/sign-up/*	SignUpPage (inline Clerk SignUp)	routing=path, fallback /onboarding
/onboarding	Onboarding (16k, 2-step)	Step1 identity + college; Step2 bio/portfolio/socials → PATCH /users/me
/dashboard	Dashboard (9.8k)	Stats GET /dashboard/stats, activity feed, quick actions
/discover	Discover (12.6k)	Browse/search/filter public projects GET /projects
/discover/colleges	DiscoverColleges (9.1k)	List colleges, join-request, register new
/discover/events	DiscoverEvents (13k)	Hackathons/workshops + inline/external register
/clubs	ClubDirectory (useListClubs)	Published clubs grid
/clubs/register	ClubRegister (in clubs.tsx 19k)	POST /clubs + Uppy useUpload
/clubs/:clubId	ClubProfile	GET /clubs/:id + members/events
/clubs/:clubId/admin	ClubAdmin	Owner edit PATCH/DELETE, members + events CRUD
/colleges/:collegeId	CollegePage (18k)	Overview/members/projects/meetings; join/moderator-apply/schedule
/projects/:projectId	ProjectPage (27k, largest)	Description/members/chat (polling, members-only)/calendar/invite/apply
/profile/me, /profile/:userId	ProfilePage (15k)	Avatar/bio/portfolio/socials, self-edit
/notifications	NotificationsPage (7.1k)	Grouped by date, mark-read/all-read
/admin	AdminPage (12.6k)	Approve colleges + moderators, GET /admin/stats
*	NotFound (711B)	404

Layout & shared

components/layout.tsx (173 lines): `AppLayout{children,userRole}` — desktop fixed sidebar w-64 + mobile drawer; `navItems` Dashboard, Discover, Colleges, Events, Clubs, Notifications (+unread Badge via `useGetUnreadNotificationCount`), My Profile, Admin (`adminOnly`); `UserButton`, next-themes Sun/Moon toggle, active via `useLocation`. components/ui/* ~60 shadcn new-york files (accordion, alert-dialog, avatar, badge, breadcrumb, button-group, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown, empty, field, form, hover-card, input+otp+group, item, kbd, label, menubar, nav-menu, pagination, popover, progress, radio, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, spinner, switch, table, tabs, textarea, toast/toaster, toggle-group, tooltip). hooks/use-toast.ts + use-mobile.tsx. lib/utils.ts `cn()=twMerge(clsx())`. lib/clerk-appearance.ts shadcn `baseTheme`, `cssLayerName` clerk.

Vite config: plugins react + Replit cartographer/dev-banner/runtime-error-modal (REPL_ID-gated), @tailwindcss/vite optimize:false; server host 0.0.0.0, port=PORT (throw if missing, dev 21903), base=BASE_PATH (throw if missing, /); build outDir dist/public; components.json style new-york, baseColor neutral, aliases @/*.

5. Backend — api-server (Express 5)

Files: `src/index.ts` (boot, PORT required, default 8080 noted in `replit.md`), `src/app.ts` (pipeline), `src/lib/{logger.ts` (pino, `LOG_LEVEL`, pretty in dev), `objectStorage.ts`, `objectAcl.ts`}, `src/middlewares/{auth.ts`, `clerkProxyMiddleware.ts`}, `src/routes/{index.ts`, `health.ts`, `users.ts`, `colleges.ts`, `clubs.ts`, `projects.ts`, `events.ts`, `notifications.ts`, `dashboard.ts`, `admin.ts`, `storage.ts`}. Pipeline: `pinoHttp` (redacted) → `CLERK_PROXY /api/_clerk` → `cors{credentials, origin:true}` → `express.json/urlencoded` → `clerkMiddleware(publishableKeyFromHost)` → `/api` router. Auth helpers: `requireAuth` (`getAuth.userId` else 401), `requireProfile` (+404), `requireAdmin` (+403 if `role!=admin`).

All routes are prefixed `/api` in `app.ts`. Storage router uses absolute `/storage/*` paths. Total ~64 operations.

Route (prefix <code>/api</code>)	Auth	Purpose
GET <code>/healthz</code>	public	Liveness probe (prod startup)
GET <code>/users/me</code> PATCH <code>/users/me</code>	requireAuth	Own profile; first <code>harshavardhankalvir2808@gmail.com</code> → admin
GET <code>/users/search?q>=2</code>	requireAuth	Name/email search for invites
GET <code>/users/:userId</code>	public	Public profile
GET <code>/colleges?search</code> POST <code>/colleges</code>	public auth	List approved; register → pending
GET <code>/colleges/:id</code> GET <code>.../members</code> GET <code>.../projects</code>	public	Detail + counts, members, projects
POST <code>.../join</code> GET <code>.../join-requests</code> PATCH <code>.../join-requests/:rid</code>	auth (moderator check)	Join request lifecycle → adds member on approve
POST <code>.../moderator-applications</code> GET <code>...</code> PATCH <code>.../:aid</code>	auth / admin	Moderator application lifecycle
PATCH <code>.../members/:uid/role</code>	auth	member/moderator switch
GET <code>.../meetings</code> POST <code>.../meetings</code>	auth (moderator to create)	College meetings
GET <code>/clubs</code> POST <code>/clubs</code>	public auth	Published clubs; register club
GET <code>/clubs/:id</code> PATCH DELETE (204)	public owner	Club CRUD + counts
GET <code>.../members</code> POST PATCH <code>:mid</code> DELETE <code>:mid</code>	mixed	Team CRUD
GET <code>.../events</code> POST PATCH <code>:eid</code> DELETE <code>:eid</code>	mixed	Club management events CRUD
GET <code>/projects?search&status;&open;</code> POST	public auth	Discover + create
GET <code>/projects/:id</code> PATCH DELETE	public owner/mod	Project CRUD
GET <code>.../members</code> POST <code>.../invites</code> POST <code>.../apply</code>	mixed	Direct invite (→accepted) + open apply
GET <code>.../applications</code> PATCH <code>.../:aid</code>	auth	Application approve/reject
GET <code>.../messages?limit&before;</code> POST <code>.../messages</code>	member only	Chat history + send (polling frontend)
GET <code>.../events</code> POST <code>.../events</code>	auth	Project calendar
GET <code>/events?type&collegeld;</code> POST GET <code>/:id</code>	public mod/admin	Global events catalogue
POST <code>/events/:id/register</code> GET <code>.../registrations</code>	auth	Event registration (201)
GET <code>/notifications</code> GET <code>.../unread-count</code>	auth	Own feed + {count} badge
PATCH <code>.../:nid/read</code> PATCH <code>.../read-all</code>	auth	Mark one / all read
GET <code>/dashboard/stats</code> GET <code>/dashboard/activity</code>	auth	<code>myProjectsCount</code> , <code>myCollegeld</code> , <code>unread</code> , <code>upcomingMeetings</code> , <code>openProjects...</code>
GET <code>/admin/college-registrations</code> PATCH <code>.../:cid</code>	requireAdmin	Approve/reject colleges + reason
GET <code>/admin/moderator-applications</code> PATCH <code>.../:aid</code>	requireAdmin	Approve/reject moderators
GET <code>/admin/users?search</code> GET <code>/admin/stats</code>	requireAdmin	All users; totals + pendings
POST <code>/storage/uploads/request-url</code>	getAuth 401	Mint GCS presigned PUT 900s → {uploadURL, objectPath}
GET <code>/storage/public-objects/*</code> GET <code>/storage/objects/*</code>	public (+ACL stub)	Serve PUBLIC paths + PRIVATE_OBJECT_DIR

6. Database Schema — 17 Tables (PostgreSQL + Drizzle)

Source: `lib/db/src/schema/index.ts` re-exporting `users.ts`, `colleges.ts`, `clubs.ts`, `projects.ts`, `events.ts`, `notifications.ts`. Connection: `pg.Pool(DATABASE_URL) → drizzle(pool,{schema});` throws if missing. `drizzle.config.ts` dialect `postgresql`, schema path above. IDs use `generatedAlwaysAsIdentity()` — auto-excluded from insert schemas, never put `id` in `.omit({})` (else 'Unrecognized key').

Table	PK / FK	Key columns
users	clerk_id TEXT PK, email UNIQUE	name, email, age INT, course, semester INT, college TEXT, pronouns, bio, avatar_url, role=user, portfolio_projects JSONB[{title,url,desc}], socials JSONB{linkedin,github,instagram,facebook,reddit,whatsapp}, timestamps
colleges	id IDENTITY PK, name UNIQUE	name, location, description, logo_url, website, status=pending approved rejected, registered_by, timestamps
college_members	id PK, college_id→colleges	college_id, clerk_id, role=member moderator admin, custom_role, joined_at
college_join_requests	id PK, college_id→colleges	college_id, clerk_id, status=pending, created_at
moderator_applications	id PK, college_id→colleges	college_id, clerk_id, motivation, status=pending, created_at
college_meetings	id PK, college_id→colleges	college_id, title, description, meet_link, scheduled_at, created_by, created_at
clubs	id PK, college_id→colleges	college_id, name, description, logo_path, brochure_path/name, created_by, status=published, timestamps
club_members	id PK, club_id→clubs	club_id, name, role, display_order=0, created_at
club_management_events	id PK, club_id→clubs	club_id, title, scheduled_at, description, banner_path, created_by, timestamps
projects	id PK, college_id→colleges?	title, description, tech_stack, status=planning, visibility=public, owner_id, college_id?, is_joint BOOL, partner_colleges JSONB[], open_for_applications BOOL, required_roles JSONB[{id,role,desc}]
project_members	id PK, project_id→projects	project_id, clerk_id, role=member, joined_at
project_applications	id PK, project_id→projects	project_id, clerk_id, applied_role, message, status=pending
project_messages	id PK, project_id→projects	project_id, clerk_id, content, created_at
project_events	id PK, project_id→projects	project_id, title, description, meet_link, scheduled_at, created_by
club_events	id PK, college_id→colleges?	title, description, type=hackathon workshop seminar other, visibility=public, college_id?, start/end_date, registration_link, max_participants, created_by
event_registrations	id PK, event_id→club_events	event_id, clerk_id, registered_at
notifications	id PK	clerk_id, type, message, link_url, read=false, created_at

7. API Contract — OpenAPI + Codegen

File lib/api-spec/openapi.yaml (1963 lines, v0.1.0, openapi 3.1.0, servers [/api]). 47 paths, 64 ops: GET×31, POST×16, PATCH×13, DELETE×4. Tags health(1), users(4), colleges(15), clubs(13), projects(14), events(6), notifications(4), dashboard(2), admin(6). Security clerk{http bearer}; mutating + personal reads secured, catalogue GETs public. ~45 schemas: HealthStatus, UserProfile(+Update, SocialLinks, PortfolioProject), College(+Input, Member, JoinRequest, ModeratorApplication, Meeting), Club(+Input/Update, Member, ManagementEvent, StorageUpload), Project(+Input/Update, Member, Invite, Application, Message, Event, RequiredRole), ClubEvent(+Input), EventRegistration, Notification, UnreadCount, DashboardStats, ActivityItem, StatusUpdate, RoleUpdate, AdminStats.

```
Codegen orval.config.ts forces title Api → lib/api-client-react/src/generated (react-query, mode
split, baseUrl /api, mutator custom-fetch.ts#customFetch) + lib/api-zod/src/generated/api.ts (zod,
coerce query/param/body/response, useDates/useBigInt). Command: pnpm --filter @workspace/api-spec run
codegen (must run after every spec edit, then pnpm -w typecheck:libs). Never hand-edit
generated/api.ts.
```

8. Authentication & Authorization (Clerk)

Layer	Implementation
Frontend @clerk/react 6 + @clerk/themes	ClerkProvider publishableKeyFromHost(hostname, VITE_CLERK_PUBLISHABLE_KEY) auto dev/prod; proxyUrl VITE_CLERK_PROXY_URL; appearance clerkAppearance (shadcn baseTheme, cssLayerName clerk, indigo primary, logo origin+base/logo.svg); SignIn/SignUp routing=path; useAuth/useUser/UserButton; routerPush/Replace strip BASE_URL; no OTP — Clerk email/password only
Backend @clerk/express 2 + @clerk/shared	clerkMiddleware((req)=>publishableKeyFromHost(getClerkProxyHost(req), CLERK_PUBLISHABLE_KEY)); requireAuth/requireProfile/requireAdmin via getAuth(req).userId + usersTable role check; storage.ts inline getAuth 401
Proxy (custom domains)	middlewares/clerkProxyMiddleware.ts: CLERK_PROXY_PATH /api/__clerk → https://frontend-api.clerk.dev, prod-only (NODE_ENV==production + CLERK_SECRET_KEY), selfHandleResponse fixes chunked Content-Length, forwards Clerk-Proxy-Url/Secret-Key/X-Forwarded-For; getClerkProxyHost prefers x-forwarded-host; mounted BEFORE express.json
Admin bootstrap	users.ts: harshavardhankalvir2808@gmail.com → role:admin on first POST/PATCH /users/me

9. Object Storage & File Uploads

Backend lib/objectStorage.ts + objectAcl.ts (ACL stub commented) + routes/storage.ts. POST /storage/uploads/request-url mints GCS presigned PUT (900s) → {uploadURL, objectPath, metadata}. GET /storage/public-objects/* serves PUBLIC_OBJECT_SEARCH_PATHS buckets; GET /storage/objects/* serves PRIVATE_OBJECT_DIR via normalizeObjectEntityPath. Frontend lib/object-storage-web (ObjectUploader.tsx, use-upload.ts, Uppy aws-s3/core/dashboard/react v5) used in ClubRegister (logo/brochure) and profile avatar. Env PUBLIC_OBJECT_SEARCH_PATHS, PRIVATE_OBJECT_DIR required when storage used.

10. Styling, Design System & UI Kit

Tailwind v4 (@tailwindcss/vite optimize:false, @import tailwindcss + tw-animate-css, @plugin typography, @custom-variant dark, @theme inline mapping HSL vars → bg-background/text-foreground/border-border). shadcn new-york (components.json: css src/index.css, baseColor neutral, cssVariables, aliases). cn() = twMerge(clsx()). Theme next-themes attribute=class default light; light :root / .dark HSL tokens; brand indigo primary 245 80% 58% (dark 245 75% 62%); fonts Inter/JetBrains Mono/Georgia; radius 0.625rem; hover-elevate/active-elevate utilities; clerk CSS layer. Icons lucide-react, motion framer-motion + tw-animate, charts recharts, forms react-hook-form+zod, dates date-fns + react-day-picker, toasts sonner + ui/toaster, Radix dialogs/drawers. Gotcha: never import Lucide icon AND define local SVG with same name (Babel duplicate declaration).

11. Configuration, Env Vars, Ports & Commands

Variable	Where	Purpose
DATABASE_URL	lib/db + drizzle.config	REQUIRED — Postgres pool, throws if missing
CLERK_SECRET_KEY	clerkProxyMiddleware	Prod proxy Clerk-Secret-Key only
CLERK_PUBLISHABLE_KEY	api-server/app.ts	Backend clerkMiddleware
VITE_CLERK_PUBLISHABLE_KEY	clubhouse/App.tsx	REQUIRED — throws if missing
VITE_CLERK_PROXY_URL	clubhouse/App.tsx	Optional → /api/__clerk
SESSION_SECRET	replit.md runbook	Legacy session docs
PORT	api index.ts + vite.config	REQUIRED — 8080 API, 21903 web
BASE_PATH / BASE_URL	vite.config + App.tsx	Vite base (/) + router stripping
NODE_ENV, LOG_LEVEL	logger.ts, proxy	pino pretty vs JSON, proxy enable
PUBLIC_OBJECT_SEARCH_PATHS, PRIVATE_OBJECT_DIR	objectStorage.ts	GCS buckets when storage used
REPL_ID	vite.config	Replit-only cartographer/banner

```
Commands (replit.md runbook): pnpm --filter @workspace/api-server run dev (PORT 8080 build+start);  
pnpm --filter @workspace/clubhouse run dev (PORT 21903 BASE_PATH=/ vite --host 0.0.0.0); pnpm run  
typecheck (tsc --build libs + artifacts/scripts); pnpm run build (typecheck + pnpm -r build); pnpm  
--filter @workspace/api-spec run codegen (after spec edits); pnpm --filter @workspace/db run push  
(dev only, drizzle-kit push); pnpm --filter @workspace/scripts run hello (tsx demo).
```

12. Build, Typecheck, Deploy (Replit)

Typecheck: root typecheck:libs (tsc --build) + per-package -p tsconfig --noEmit for artifacts/scripts. API build node ./build.mjs (esbuild ESM bundle → dist/index.mjs, externals native/cloud SDKs, esbuild-plugin-pino); start node --enable-source-maps dist/index.mjs. Web build vite build → dist/public. .replit modules nodejs-24 + postgresql-16, router=application, autoscale; artifact.toml declares dev PORT/BASE_PATH, prod static rewrites /*→/index.html (web) and health /api/healthz (api). Per-package tsconfigs: clubhouse (noEmit, jsx preserve, dom, @/* paths, ref api-client-react); api-server (outDir dist, rootDir src, ref db+api-zod); libs composite declarationMap emitDeclarationOnly.

13. User Flows & Product Features

Onboarding (2-step): Clerk sign-up → /onboarding Step1 (name/age/course/semester/college/pronouns) → Step2 (bio/portfolio/socials) → PATCH /users/me (admin auto-grant for hardcoded email). Dashboard: stats + activity + quick actions. Discover Projects: GET /projects?search&status;&open;, apply (POST .../apply) or create. Discover Colleges: GET /colleges, POST join (→pending), POST new college (→pending admin). Discover Events: GET /events?type&collegeld;, inline register or external link. College page: overview/members/projects/meetings; moderator schedules meetings. Project page (richest): description/members/chat (GET/POST messages, members-only polling)/calendar/invite via GET /users/search + POST invites/apply. Clubs: directory/register/profile/admin (owner CRUD + members/events). Profile: avatar/bio/portfolio/socials self-edit. Notifications: grouped by date, unread badge, mark-read/all-read. Admin: college-registrations + moderator-applications approve/reject + stats + users search.

14. Roles & Permissions Matrix

Capability	Guest	Member	Moderator	Admin (platform)
Browse colleges/projects/events	Yes	Yes	Yes	Yes
Register college / create project/event	No	Yes (event needs mod)	Yes	Yes
Join college / apply project / register event	No	Yes	Yes	Yes
Project chat + calendar	No	If member	If member	If member / override
Schedule college meeting	No	No	Yes	Yes
Approve join-requests / change roles	No	No	Yes (own college)	Yes
Club owner edit (PATCH/DELETE clubs)	No	Owner only	Owner only	Yes
Approve colleges / moderators, view admin stats	No	No	No	Yes (requireAdmin)

15. Architecture Decisions & Gotchas

Clerk for auth (publishableKeyFromHost supports dev+prod automatically; proxy fixes .replit.app custom-domain cookie issues). Contract-first API (Orval → hooks + Zod, zero hand fetch). generatedAlwaysAsIdentity IDs auto-excluded — never include id in .omit({}). Admin hardcoded email. No OTP (Clerk email/password only). Always regen codegen after spec change before editing pages. No duplicate Lucide + local SVG names. Clerk routerPush/Replace must strip base. API listens 8080 via PORT env (not 5000). pnpm minimumReleaseAge 1440 must stay on. CORS origin:true + credentials. Pino redaction for sensitive headers.

16. Repository Health & Git History

Git log (recent): 7864275 Published your App; aeec308 Add application logo; c3fbd69 Published your App; ffb960c Update api client definitions; f283d71 Implement project management + profile updates; 8b9d10a Published your App; dec418c Add user + college management; 185439b Initial commit. .npmrc + preinstall enforce pnpm (delete package-lock/yarn.lock). attached_assets holds reference image + pasted website text. .agents/skills document pnpm-workspace + clerk-auth wiring. No test suite detected — verification is via typecheck + build.

17. Roadmap & Improvement Recommendations

1) Add automated tests (Vitest for hooks/pages, Supertest for routes, drizzle seed). 2) Replace chat polling with WebSocket/SSE for true real-time. 3) Add pagination + cursor (messages, projects, colleges) — currently limit/before only on messages. 4) Harden RBAC: move moderator/owner checks into reusable middleware (currently partly inline). 5) Enable object ACL enforcement (currently stubbed public). 6) Add rate-limiting + audit log for admin actions. 7) CI: pnpm typecheck + build + codegen drift check (fail if generated/ differs). 8) Observability: structured pino + request IDs + /healthz DB check. 9) Accessibility + mobile nav polish (layout already has drawer). 10) Document .env.example to prevent missing-env throws.

Appendix A — Directory Tree & Cheat Sheet

```
D:\OpencodeProjs\Clubhouse\Web-Mobile-Fit\ ■■■ .agents/ .git/ .npmrc .replit .replitignore ■■■
attached_assets/ ■■■ artifacts/api-server/src/{app.ts,index.ts,lib/,middlewares/,routes/} ■■■ artif
acts/clubhouse/src/{App.tsx,main.tsx,index.css,lib/,hooks/,components/{layout.tsx,ui/*},pages/*13}
■■■ artifacts/mockup-sandbox/src/ ■■■
lib/{api-client-react,api-spec/openapi.yaml,api-zod,db/schema,object-storage-web} ■■■
scripts/src/hello.ts ■■■ package.json pnpm-workspace.yaml tsconfig.* replit.md

Cheat sheet: dev API `pnpm --filter @workspace/api-server run dev`; dev web `pnpm --filter
@workspace/clubhouse run dev`; typecheck `pnpm run typecheck`; build `pnpm run build`; codegen `pnpm
```

```
--filter @workspace/api-spec run codegen`; db push `pnpm --filter @workspace/db run push`. Env:  
DATABASE_URL, CLERK_SECRET_KEY, CLERK_PUBLISHABLE_KEY, VITE_CLERK_PUBLISHABLE_KEY,  
VITE_CLERK_PROXY_URL, SESSION_SECRET, PORT, BASE_PATH.
```

— End of full documentation —